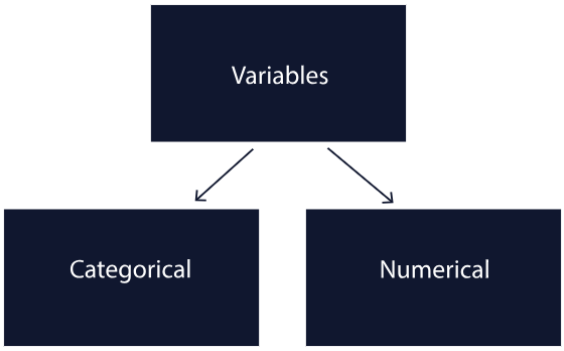
Set 24: Variable Types for Data Science

Skill 24.1: Differentiate between numerical and categorical variables

Skill 24.1 Concepts

Variables define datasets. They are the characteristics or attributes that we evaluate during data collection. There are two ways to do that evaluation: we can measure or we can categorize.

How we evaluate determines what kind of variable we have. Since there are only two ways to get data, there are only two types of variables: numerical and categorical.



Every observation (the individuals or objects we are collecting data about) is classified according to its characteristics. In "flat" file formats (like tables, csvs, or DataFrames), the observations are the rows, the variables are the columns, and the values are at the intersection.

Typically, the best way to understand your data is to look at a sample of it.

In the example dataset about cereal below, we can look at the first few rows with the *head()* method to get an idea of the variable types that we have.

cereal

	id	name	mfr	type	fiber	rating	shelf	vitamins	coupons	price
0	22341	100% Bran…	Nestle	C	10.0	68.40	top	25	4	3.46
1	22791	100% Natur…	Quaker Oats	C	2.0	33.98	top	0	1	3.36
2	98141	All-Bran…	Kelloggs	C	9.0	59.43	top	25	4	2.07
3	20001	All-Bran w…	Kelloggs	C	14.0	93.70	top	25	3	3.57
4	67121	Almond Del…	Ralston P..	C	1.0	34.38	top	25	1	5.21

There are several types of variables. For example:

- The *price* column describes how much the cereal costs. We don't know if that's how much the consumer pays or the grocer pays, but we can be fairly sure that it's a numerical variable.
- In the *mfr* column, there are labels like Nestle, Quaker Oats, and Kelloggs, which seem like brands. Since brands are categories, mfr is most likely a categorical variable.
- The *id* column also has numbers, but we can assume that since it's the id, it's not actually representing a value. It's probably the label for the observation. Since it's a label, even though it's a number, id is a categorical variable.

If we were downloading this from a data repository, we would expect a data dictionary to define these variables and validate (or invalidate) our assumptions.

It is still important to inspect our dataset because it gives us a better understanding of the data that we are working with and the kinds of operations that will be possible.

Skill 24.1 Exercise 1

Skill 24.2: Identify the 3 types of categorical variables

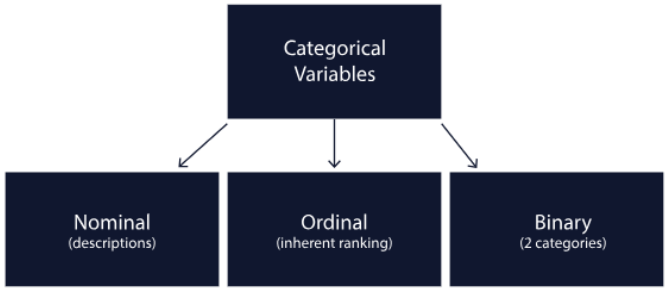
Skill 24.2 Concepts

Without getting too philosophical about it, "your mind is a categorization machine"

We move through the world by categorizing things into various groups: safe/unsafe, best/worst, on/off. These categorizations help us process information. They also create major problems for us when we over-categorize, and can lead to bias and unfair assumptions.

Categorical variables come in 3 types,

- Nominal variables, which describe something,
- Ordinal variables, which have an inherent ranking, and
- Binary variables, which have only two possible variations.



Nominal Variables

When we want to describe something about the world, we need a nominal variable. Nominal variables are usually words (i.e., red, yellow, blue or hot, cold), but they can also be numbers (i.e., zip codes or user id's).

Often, nominal variables describe something with a lot of variation. It can be hard to capture all of that variation, so an 'Other' category is often necessary. For example, in the case of color, we could have a lot of different labels, but might still need an 'Other' category to capture anything we missed.

Ordinal Variables

When our categories have an inherent order, we need an ordinal variable. Ordinal variables are usually described by numbers like 1st, 2nd, 3rd. Places in a race, grades in school, and the scales in survey responses (Likert Scales) are ordinal variables

Ordinal variables can be a little tricky because even though they are numbers, it doesn't make sense to do math on them. For example, let's say an Olympian won a Gold medal (1st place) and a Bronze medal (3rd place). We wouldn't say that they averaged Silver medals (2nd place).

Binary Variables

When there are only two logically possible variations, we need a binary variable. Binary variables are things like on/off, yes/no, and TRUE/FALSE. If there is any possibility of a third option, it is not a binary variable.

Let's return to the cereal dataset,

cereal

	id	name	mfr	type	fiber	rating	shelf	vitamins	coupons	price
0	22341	100% Bran…	Nestle	C	10.0	68.40	top	25	4	3.46
1	22791	100% Natur…	Quaker Oats	C	2.0	33.98	top	0	1	3.36
2	98141	All-Bran…	Kelloggs	C	9.0	59.43	top	25	4	2.07
3	20001	All-Bran w…	Kelloggs	C	14.0	93.70	top	25	3	3.57
4	67121	Almond Del…	Ralston P..	C	1.0	34.38	top	25	1	5.21

There are some obvious categorical variables: The name of the product, the mfr (manufacturer), and the shelf are all nominal categorical variables. We know this because they are written in descriptive words or letters.

A little less obvious is the type field. They are all 'C', which could be a ranking (A, B, C, and therefore an ordinal variable) or it could be a description and therefore a nominal variable. We would have to return to the data dictionary to find out for certain.

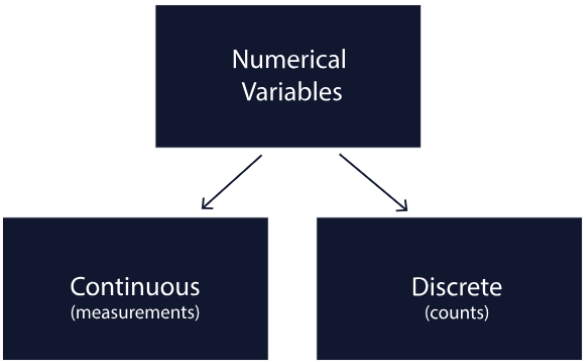
The id field may also cause confusion. It's a number, but it's not a count or a measurement. Rather, 'id' is a categorical variable since it is describing each observation in the same way that the name is.

Skill 24.2 Exercise 1

Skill 24.3: Identify the 2 types of numerical variables

Skill 24.3 Concepts

Numerical variables are created two ways: through measurement and counting.



Continuous variables come from measurements. For a variable to be continuous, there must be infinitely smaller units of measurement between one unit and the next unit. Continuous variables can be represented by decimal places (but because of rounding, sometimes they are whole numbers). Length, time, and temperature are all good examples of continuous variables because they all increase continuously.

Discrete variables come from counting. For a variable to be discrete, there must be gaps between the smallest possible units. People, cars, and dogs are all good examples of discrete variables.

Some variables depend on context to determine if they are continuous or discrete. Money and time can both be measured continuously or discretely.

For money, all currencies have a smallest-possible-unit (i.e., the cent in USD) and are therefore discrete. However, banks and other institutions sometimes measure money in fractions of a cent, treating it like a continuous variable.

It is therefore always essential to understand how your data was created in order to represent it appropriately.

Let's take a look at the cereal dataset again.

cereal

	id	name	mfr	type	fiber	rating	shelf	vitamins	coupons	price
0	22341	100% Bran…	Nestle	C	10.0	68.40	top	25	4	3.46
1	22791	100% Natur…	Quaker Oats	C	2.0	33.98	top	0	1	3.36
2	98141	All-Bran…	Kelloggs	C	9.0	59.43	top	25	4	2.07
3	20001	All-Bran w…	Kelloggs	C	14.0	93.70	top	25	3	3.57
4	67121	Almond Del…	Ralston P..	C	1.0	34.38	top	25	1	5.21

There are five numerical variables: fiber, rating, vitamins, coupons, and price.

Without looking at the data dictionary, we can make some guesses about what kind of numerical variables they are:

Fiber, rating, and price all have decimal places. That's our first clue that they might be continuous. Based on our limited knowledge, we might guess that fiber and rating are both continuous measurements that could have more decimal places, and price is discrete because there's nothing smaller than a cent.

Vitamins and coupons do not have decimal places. Vitamins and coupons both seem like good candidates to be counts and therefore discrete. The answers to "how many vitamins" and "how many coupons" would both be whole numbers. (We already said that ID is categorical)

We would be more confident in our answers if we were able to inspect the documentation. But sometimes documentation isn't available and you have to take your best guess.

Skill 24.3 Exercise 1

Skill 24.4: Convert between numerical data types

Skill 24.4 Concepts

When you read a data file (such as a csv) with pandas, data types are assigned to each column. Pandas does its best to predict what kind of data type each variable should contain. For example, if a column contains only integer values, it will be stored as an int32 or int64. This usually works, but problems can arise for our analysis later on when there's a mismatch between the real-world variable type and the data type pandas assigns.

With numerical variables, pandas expects any column that has decimal values to be a float and anything without decimal values to be an integer. If any non-numeric characters appear in the column, pandas will treat it as an object.

It's possible to determine the data types of the columns in your DataFrame with the *dtypes* attribute.

Data

	id	name	mfr	type	fiber	rating	shelf	vitamins	coupons	price
0	22341	100% Bran…	Nestle	C	10.0	68.40	top	25	4	3.46
1	22791	100% Natur…	Quaker Oats	C	2.0	33.98	top	0	1	3.36
2	98141	All-Bran…	Kelloggs	C	9.0	59.43	top	25	4	2.07
3	20001	All-Bran w…	Kelloggs	C	14.0	93.70	top	25	3	3.57
4	67121	Almond Del…	Ralston P..	C	1.0	34.38	top	25	1	5.21

Code

```
print(cereal.dtypes)
```

Output

```
name                                    object
id                                      int64
name                                    object
mfr                                     object
type                                    object
fiber                                   float64
rating                                  string
shelf                                   object
vitamins                                int64
coupons                                 int64
price                                   string
dtype: object
```

Best practices for data storage say that we should match the data type of the column with its real-world variable type. Therefore,

- Continuous (numerical) variables should usually be stored as the float data type because they allow us to store decimal values.
- Discrete (numerical) variables should be stored as the int datatype to represent mathematically that they are discrete

Using float and int to store quantitative variables is important so that you can later perform numerical operations on those values. It also helps indicate what the variables refer to in the real world. Keeping them separate helps ensure that we perform the right calculations and get the right results.

If a variable appears with the wrong data type, we can change it with the *astype()* function.

Data

name	**object**
id	int64
name	object
mfr	object
type	object
fiber	float64
rating	string *change to float
shelf	object
vitamins	int64
coupons	int64
price	string *change to float
dtype: object	

Code

```
cereal['rating'] = cereal['rating'].astype('float64')
cereal['price'] = cereal['price'].astype('float64')
print(cereal.dtypes)
```

Output

name	**object**
id	int64
name	object
mfr	object
type	object
fiber	float64
rating	float64
shelf	object
vitamins	int64
coupons	int64
price	float64
dtype: object	

The *astype()* function can be used to convert between data types, including,

- int32, int64
- float32, float64
- object
- string
- bool
- category

However, some data types require all values to be filled in. For example, you cannot convert between a float and an int if there are any null values.

Skill 24.4 Exercise 1

Skill 24.5: Convert between categorical data types

Skill 24.5 Concepts

Now let's focus on Categorical variables and make sure they are in the correct format. Let's take another look at the cereal dataset to assess the data types of our categorical variables.

Data	
name	**object**
id	int64
name	object
mfr	object
type	object
fiber	float64
rating	float64
shelf	Object
vitamins	int64
coupons	int64
price	float64
dtype: object	

Just like with numerical variables, best practices for categorical data storage say that we should match the data type of the column with its real-world variable type. However, the types are a little more nuanced,

- **Nominal variables** represent labels with no inherent order. In pandas, these are best stored as either the *string* data type (for text data) or the *category* data type (when values repeat and grouping is common).

- **Ordinal variables** represent categories with a meaningful order but unequal spacing. In pandas, ordinal variables should be stored as *category* data, not as numeric types or object.

The *object* data type can hold any Python object (strings, integers, booleans, lists, etc.), which makes it flexible but inefficient and ambiguous. Because of this, string operations are not guaranteed to work reliably on object columns unless the values are consistently strings.

The *string* data type is specifically designed for text. If you plan to perform string operations such as *lower()* or *contains(),* you should explicitly convert the column to string using .astype("string"), since pandas often defaults to *object* when reading data.

Let's look at some examples below,

Data

name	object
id	int64 *change to string
name	Object *change to string
mfr	Object *change to string
type	object
fiber	float64
rating	float64
shelf	object *change to category
vitamins	int64
coupons	int64
price	float64
dtype: object	

Code

```
cereal['id'] = cereal['id'].astype("string")
cereal['name'] = cereal['name'].astype("string")
cereal['mfr'] = cereal['mfr'].astype("string")
cereal['shelf'] = cereal['shelf'].astype("category")
```

Output

name	object
id	string
name	string
mfr	string
type	object
fiber	float64
rating	float64
shelf	category
vitamins	int64
coupons	int64
price	float64
dtype: object	

Skill 24.5 Exercise 1

Skill 24.6: Set and apply the order of a categorical variable

Skill 24.6 Concepts

We saw in a previous example that we can set a variable to a category data type. At this point, however, Python does not know that these categories have an inherent order.

Data

	id	name	mfr	type	fiber	rating	shelf	vitamins	coupons	price
0	22341	100% Bran…	Nestle	C	10.0	68.40	top	25	4	3.46
1	22791	100% Natur…	Quaker Oats	C	2.0	33.98	top	0	1	3.36
2	98141	All-Bran…	Kelloggs	C	9.0	59.43	top	25	4	2.07
3	20001	All-Bran w…	Kelloggs	C	14.0	93.70	top	25	3	3.57
4	67121	Almond Del…	Ralston P..	C	1.0	34.38	top	25	1	5.21

Code

```
display(cereal.shelf.unique)
```

Output

Categories (3, object): ['bottom', 'mid', 'top']

The order of a category variable can be set with the *set_categories()* method.

Data

	id	name	mfr	type	fiber	rating	shelf	vitamins	coupons	price
0	22341	100% Bran…	Nestle	C	10.0	68.40	top	25	4	3.46
1	22791	100% Natur…	Quaker Oats	C	2.0	33.98	top	0	1	3.36
2	98141	All-Bran…	Kelloggs	C	9.0	59.43	top	25	4	2.07
3	20001	All-Bran w…	Kelloggs	C	14.0	93.70	top	25	3	3.57
4	67121	Almond Del…	Ralston P..	C	1.0	34.38	top	25	1	5.21

Code

```
shelf_order = ['bottom', 'mid', 'top']

cereal["shelf"] = cereal["shelf"].cat.set_categories(shelf_order, ordered=True)

display(cereal.shelf.unique)
```

Output

Categories (3, object): ['bottom' < 'mid' < 'top']

The code above sorts cereals from **top → mid → bottom**, instead of alphabetical order which allows for meaningful comparisons. For example, the code below returns the serials on the top shelf.

```
display(cereal[cereal["shelf"] > "mid"])
```

Skill 24.6 Exercise 1

Skill 24.7: Apply one-hot encoding to nominal categorical variables

Skill 24.7 Concepts

In the previous exercise, we saw how label encoding can be useful for ordinal categorical variables. But sometimes we need a different approach. This could be because,

- We have a nominal categorical variable (animal species), so it doesn't really make sense to assign numbers like 0,1,2,3,4,5 to our categories, as this could create an order among the species that is not present. For example, the following implies that lizard > dog,

```
dog = 0
cat = 1
bird = 2
fish = 3
lizard = 4
```

- We have an ordinal categorical variable but we don't want to assume that there's equal spacing between categories. In the example below, the difference in age restriction between G and PG is not the same as the difference between PG-13 and R. So, while the order matters, the distance between categories does not.

```
G → PG → PG-13 → R → NC-17
```

One-hot encoding (OHE) is useful because it lets models work with categorical data without accidentally adding meaning that isn't there.

With OHE, we essentially create a new binary variable for each of the categories within our original variable. This technique is useful when managing nominal variables because it encodes the variable without creating an order among the categories.

To perform OHE on a variable within a pandas dataframe, we can use the pandas *get_dummies()* method which creates a binary or "dummy" variable for each category. We can assign the columns to be encoded in the columns parameter, and set the data parameter to the dataset we intend to alter. The *pd.get_dummies()* method will also work on data types other than category.

Data

	id	name	mfr	type	fiber	rating	shelf	vitamins	coupons	price
0	22341	100% Bran…	Nestle	C	10.0	68.40	top	25	4	3.46
1	22791	100% Natur…	Quaker Oats	C	2.0	33.98	top	0	1	3.36
2	98141	All-Bran…	Kelloggs	C	9.0	59.43	top	25	4	2.07
3	20001	All-Bran w…	Kelloggs	C	14.0	93.70	top	25	3	3.57
4	67121	Almond Del…	Ralston P..	C	1.0	34.38	top	25	1	5.21

Code

```
cereal = pd.get_dummies(data = cereal, columns = ['type'])

display(cereal.head())
```

Output

	id	name	mfr	type	fiber	rating	shelf	vitamins	coupons	price	type_C	type_H
0	22341	100% Bran…	Nestle	C	10.0	68.40	top	25	4	3.46	True	False
1	22791	100% Natur…	Quaker Oats	C	2.0	33.98	top	0	1	3.36	True	False
2	98141	All-Bran…	Kelloggs	C	9.0	59.43	top	25	4	2.07	True	False
3	20001	All-Bran w…	Kelloggs	C	14.0	93.70	top	25	3	3.57	True	False
4	67121	Almond Del…	Ralston P..	C	1.0	34.38	top	25	1	5.21	True	False

By passing in the dataset and column that we want to encode into *pd.get_dummies(),* we have created a new dataframe that contains two new binary variables with values True (1) and False (0). It is important to note that OHE works best when we do not create too many additional variables, as increasing the dimensionality of our dataframe can create problems when working with certain machine learning models.

Skill 24.7 Exercise 1